

LAPORAN TUGAS BESAR TEORI BAHASA FORMAL DAN AUTOMATA

Milestone 2 - Syntax Analysis



Oleh

13524014 - Yusuf Faishal Listyardi

13524046 - Farrel Limjaya

13524066 - Nathanael Gunawan

13524070 - A. Fawwaz Azam Wicaksono

**PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO & INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG**

2026

DAFTAR ISI

BAB I	
PENDAHULUAN.....	1
1.1. Pengantar.....	1
1.2. Spesifikasi.....	1
1.2.1. Revisi Program Milestone.....	2
1.2.2. Spesifikasi Program.....	2
1.2.3. Daftar Node.....	2
BAB II	
DASAR TEORI.....	8
2.1. Context Free Grammar.....	8
2.2. Parser.....	8
2.3. Parser Tree.....	9
2.4. Recursive Descent.....	10
BAB III	
PERANCANGAN DAN IMPLEMENTASI.....	11
3.1. Gambaran Umum Program.....	11
3.2. Representasi Parse Tree.....	12
3.3. Struktur Direktori Program.....	14
3.4. Kelas Parser dan Mekanisme Recursive Descent.....	16
BAB IV	
PENGUJIAN.....	20
4.1. Pengujian 1.....	20
4.2. Pengujian 2.....	22
4.3. Pengujian 3.....	26
4.4. Pengujian 4.....	31
4.5. Pengujian 5.....	34
4.6. Pengujian 6.....	37
4.7. Pengujian 7.....	42
4.8. Pengujian 8.....	45
4.9. Pengujian 9.....	46
4.10. Pengujian 10.....	47
BAB V	
KESIMPULAN DAN SARAN.....	48
5.1. Kesimpulan.....	48
5.2. Saran.....	48
BAB VI	
LAMPIRAN.....	50
6.1. Lampiran Link Repository Github.....	50
6.2. Pembagian Tugas.....	50
REFERENSI.....	51

BAB I

PENDAHULUAN

1.1. Pengantar

Pada milestone I lexical analysis, source code sudah dibentuk menjadi token-token tertentu. Token-token ini akan digunakan oleh tahap selanjutnya yaitu Syntax Analysis atau sering disebut parser.

Syntax Analysis atau Parser bertugas untuk memeriksa urutan token yang dihasilkan oleh lexical analyzer agar sesuai dengan aturan tata bahasa (grammar) dari bahasa pemrograman yang dikompilasi.

Parser akan membangun struktur sintaks seperti Parse Tree atau AST (Abstract Syntax Tree) yang merepresentasikan struktur hierarkis program sumber berdasarkan grammar yang telah didefinisikan.

Tahapan Parser bertujuan untuk:

- Memastikan urutan token membentuk struktur sintaks yang valid.
- Mendeteksi dan melaporkan kesalahan sintaks (syntax error).
- Mempersiapkan representasi program untuk tahap berikutnya.

Parse Tree

Parse Tree adalah representasi lengkap dari struktur sintaks program sesuai dengan grammar bahasa yang digunakan. Setiap node di dalam parse tree merepresentasikan produksi grammar, sehingga pohon ini memuat semua simbol terminal dan non-terminal.

Parse tree berguna untuk menunjukkan bagaimana token-token hasil lexical analysis membentuk struktur program yang sesuai dengan aturan grammar.

1.2. Spesifikasi

Pada milestone 2, Anda diminta untuk membuat tahapan kedua dari compiler, yaitu syntax analysis atau parser. Parser berfungsi melakukan analisis sintaksis (syntax analysis) dengan menggunakan Recursive Descent untuk mengenali struktur gramatikal dalam deretan token.

1.2.1. Revisi Program Milestone

Terdapat beberapa revisi pada DFA yang dibuat sebelumnya untuk pengerjaan kedepannya, yaitu pembacaan separator (spasi, new-line, dan tab). Dua sekuens-karakter yang terpisah dengan separator pasti merupakan dua token yang berbeda. Sebagai contoh:

1. katasambung $\rightarrow$ ident(katasambung)
2. kata pisah $\rightarrow$ ident(kata), ident(pisah)
3. := kata $\rightarrow$ becomes, ident(kata)

Sekuens-karakter yang tidak terpisah dengan separator secara default dibaca berdasarkan prinsip longest-match. Sebagai contoh:

1. 1.23 $\rightarrow$ realcon(1.23)
2. ident:=1 $\rightarrow$ ident(ident), becomes, intcon(1)
3. .23.e-+/1.e-+/1 $\rightarrow$ unknown(.23.e-+/1.e-+/1)

1.2.2. Spesifikasi Program

Tahapan ini bertujuan untuk menganalisis struktur sintaksis dari list of tokens yang dihasilkan lexer dan membangun **Parse Tree** yang merepresentasikan hierarki struktur program sesuai dengan grammar bahasa Arion. Parse Tree ini akan menjadi masukan bagi tahap semantic analysis dalam proses kompilasi.

Masukan	List Token (misalnya VAR(X), ASSIGN(=), OPERATOR(+))
Keluaran	Parse Tree
Otomata	Recursive Descent

1.2.3. Daftar Node

Berikut merupakan daftar node yang mungkin muncul pada hasil Parse Tree.

Daftar Node				
No	Nama Node	Deskripsi	Aturan Produksi	Contoh
1	<program>	Node root yang merepresentasikan keseluruhan program Arion	program $\rightarrow$ program-header + declaration-part + compound-statement + period	Seluruh struktur program dari awal hingga akhir

2	<program-header>	Bagian kepala program yang berisi nama program	programsy + ident + semicolon	program Hello;
3	<declaration-part>	Bagian deklarasi yang dapat berisi const, type, var, procedure, function	(const-declaration)* + (type-declaration)* + (var-declaration)* + (subprogram-declaration)*	Semua deklarasi sebelum begin
4	<const-declaration>	Deklarasi konstanta	constsy + (ident + eql + constant + semicolon)+	const INT == 100; REAL == 10.0; CHAR == 'a'; STRING == 'abcd';
5	<constant>	Konstanta dapat berupa ident , intcon , realcon , charcon , atau string	charcon string [(plus minus)? + (ident intcon realcon)]	'a' 'abcd' 10 +10.0 -a
6	<type-declaration>	Deklarasi tipe data baru	typesy + (ident + eql + type + semicolon)+	type Range == 1..10; type Month == (Jan, Feb, Mar, Apr);
7	<var-declaration>	Deklarasi variabel	varsy + (identifier-list + colon + type + semicolon)+	var a, b: integer;
8	<identifier-list>	Daftar identifier yang dipisahkan koma	ident (comma + ident)*	a, b, c
9	<type>	Tipe data yang terdefinisi atau didefinisikan pengguna. Dapat berupa ident , array-type, range, atau enumerated	ident array-type range enumerated record-type	integer, array[1..10] of integer
10	<array-type>	Definisi tipe array	arraysy + lbrack + (range ident) + rbrack + ofsy + type	array[1..10] of integer array[char] of integer
11	<range>	Rentang nilai untuk array atau subrange	constant + period + period + constant	1..10, 'a'..'z'

12	<enumerated>	Nilai tertentu yang ada di dalam list	lparent + ident + (comma + ident)* + rparent	(Jan, Feb, Mar, Apr)
13	<record-type>	Tipe data template, sama dengan <i>class</i> dalam OOP	recordsy + field-list + endsy	type record-name = record field-1: field-type1; field-2: field-type2; ... field-n: field-typen; end
14	<field-list>	Penjabaran atribut/field dari record	field-part + (semicolon + field-part)*	..field-1..; ..field-2..
15	<field-part>	Singular atribut/field dari record	identifier-list + colon + type	field-1: field-type1;
16	<subprogram-declaration>	Deklarasi prosedur atau fungsi	procedure-declaration function-declaration	procedure print(x: integer);
17	<procedure-declaration>	Deklarasi prosedur	proceduresy + ident + (formal-parameter-list)? + semicolon + block + semicolon	Prosedur dengan parameter opsional
18	<function-declaration>	Deklarasi fungsi	functionsy + ident + (formal-parameter-list)? + colon + ident + semicolon + block + semicolon	Fungsi yang mengembalikan nilai
19	<block>	Bagian kode yang mengandung deklarasi dan statement	declaration-part + compound-statement	var ... begin ... end
20	<formal-parameter-list>	Daftar parameter formal	lparent + parameter-group + (semicolon + parameter-group)* + rparent	(x, y: integer; z: real)

21	<parameter-group>	Daftar parameter disertai dengan tipe data parameter	identifier-list + colon + (ident array-type)	x, y: integer
22	<compound-statement>	Blok statement yang diawali begin dan diakhiri end	beginsy + statement-list + endsy	begin ... end
23	<statement-list>	Daftar statement yang dipisahkan semicolon	statement (semicolon + statement)*	Urutan statement dalam blok
24	<statement>	Deskripsi aksi yang harus dilakukan program komputer	(assignment-statement if-statement case-statement while-statement repeat-statement for-statement procedure/function-call)?	x := 5; if x > 0 then y := 1 else y := 0 while x < 10 do x := x + 1
25	<variable>	Variable dapat berbentuk ident, elemen array, atau field record	ident + (component-variable)*	someVar someArray[1] someField
26	<component-variable>	Variable dalam bentuk element array atau field record	(lbrack + index-list + rbrack) (period + ident)	someArray[1] someRecord.someField someArray[1][2]
27	<index-list>	Penunjuk index dalam array	(intcon charcon ident) + (comma + index-list)*	[1, 2, 3] ['a', 'b'] [true, false]
28	<assignment-statement>	Statement penugasan nilai ke variabel	variable + becomes + expression	x := 5, y := x + 10
29	<if-statement>	Statement kondisional	ifsy + expression + thensy + statement + (elsy + statement)?	if x > 0 then y := 1 else y := 0
30	<case-statement>	Statement kondisional dengan banyak kondisional	casesy + expression + ofsy + case-block + endsy	case i of 0: x := 0; 1 : x := 1; end

31	<case-block>	Deklarasi case dan statement yang harus dilakukan	constant + (comma + constant)* + colon + statement + (semicolon + case-block?)*	0: x := 0; 1, 2: x := 1
32	<while-statement>	Perulangan dengan kondisi di awal	whilesy + expression + dosy + statement	while x < 10 do x := x + 1
33	<repeat-statement>	Perulangan dengan kondisi di akhir	repeatsy + statement-list + untilsy + expression	repeat x := x - 1 until x == 0
34	<for-statement>	Perulangan dengan counter	forsy + ident + becomes + expression + (tosy downtosy) + expression + dosy + statement	for i := 1 to 10 do ...
35	<procedure/function-call>	Pemanggilan prosedur atau fungsi	ident + (lparent + parameter-list? + rparent)?	writeln('Hello'), print(x, y)
36	<parameter-list>	Daftar parameter aktual saat pemanggilan	expression (comma + expression)*	'Result = ', b, 100
37	<expression>	Ekspresi yang menghasilkan nilai	simple-expression (relational-operator + simple-expression)?	x + y, a > b, 5 * (x + 1)
38	<simple-expression>	Ekspresi tanpa operator relasional	(plus minus)? term (additive-operator + term)*	x + y - z, 5 * 2
39	<term>	Bagian ekspresi dengan prioritas lebih tinggi	factor (multiplicative-operator + factor)*	x * y, a div b
40	<factor>	Unit terkecil dalam ekspresi	ident intcon realcon charcon string (lparent + expression + rparent) (notsy + factor) procedure/function-call variable	x, 5, 'a', (x+y), not a
41	<relational-operator>	Operator perbandingan	eq neq gtr geq lss leq	x == 5, y > 10

42	<additive-operator>	Operator penjumlahan/pengurangan	plus minus orsy	x + y, a or b
43	<multiplicative-operator>	Operator perkalian/pembagian	times rdiv idiv imod andsy	x * y, a div b, p and q

Program harus menggunakan Recursive Descent dengan **grammar yang di-hardcode dalam program** dan keseluruhan grammar dijelaskan kembali dalam laporan yang dibuat.

Input awal dari program **tetap merupakan source code Arion** sehingga sebelum dimasukkan ke parser, kode dimasukkan terlebih dahulu ke lexer. Output syntax analyzer merupakan parse tree yang di **print ke terminal** dan **disimpan ke dalam file txt**.

BAB II

DASAR TEORI

2.1. Context Free Grammar

Context-Free Grammar (CFG) adalah *grammar* formal yang digunakan untuk mendefinisikan struktur sintaksis suatu bahasa. Secara formal, CFG dapat dinyatakan sebagai 4-tuple $G = (V, \Sigma, S, P)$, dengan V sebagai himpunan simbol non-terminal, Σ sebagai himpunan simbol terminal, S sebagai simbol awal, dan P sebagai himpunan aturan produksi. Simbol non-terminal merepresentasikan struktur abstrak dalam bahasa, seperti *declaration*, *statement*, atau *expression*, sedangkan simbol terminal merepresentasikan token yang muncul langsung pada input.

Aturan produksi pada CFG memiliki bentuk $X \rightarrow \alpha$, dengan X merupakan simbol non-terminal dan α merupakan rangkaian simbol terminal maupun non-terminal. Melalui aturan produksi tersebut, suatu input dapat diperiksa apakah dapat dibentuk dari simbol awal grammar. Jika rangkaian token hasil lexical analysis dapat diturunkan sesuai aturan CFG, maka input dianggap valid secara sintaksis.

CFG banyak digunakan dalam perancangan bahasa pemrograman karena mampu merepresentasikan struktur sintaksis seperti deklarasi variabel, ekspresi, percabangan, perulangan, dan *statement* lainnya. Dalam kompilator, CFG menjadi dasar bagi parser untuk menentukan apakah urutan token yang diberikan oleh lexer sesuai dengan aturan bahasa yang telah didefinisikan.

2.2. Parser

Parser (atau penganalisis sintaksis) adalah komponen perangkat lunak yang bertugas menganalisis string atau teks masukan untuk memeriksa kebenaran struktur sintaksisnya berdasarkan aturan tata bahasa formal yang telah ditentukan. Dalam sebuah kompilator, parser bekerja dengan menerima token dari penganalisis leksikal (lexer) dan memeriksa apakah masukan tersebut memenuhi aturan dari context-free grammar (CFG) bahasa pemrograman terkait.

Jika sintaksnya sesuai, parser akan membangun sebuah struktur data hierarkis, seperti, pohon parse (parse tree) atau pohon sintaks abstrak (abstract syntax tree), yang merepresentasikan struktur program tersebut secara terstruktur untuk diproses ke tahapan kompilasi selanjutnya.

Parser secara umum dapat dibedakan menjadi dua pendekatan utama, yaitu top-down parsing dan bottom-up parsing. Top-down parsing memulai proses analisis dari simbol awal grammar, kemudian mencoba menurunkannya menjadi rangkaian token input. Dengan pendekatan ini, parser berusaha membuktikan bahwa input dapat dibentuk dari aturan grammar mulai dari struktur paling umum menuju struktur yang lebih kecil.

Sebaliknya, bottom-up parsing bekerja dari arah yang berlawanan. Parser memulai dari token-token input, kemudian mencoba menggabungkannya secara bertahap menjadi non-terminal yang lebih besar hingga akhirnya mencapai simbol awal grammar. Dengan kata lain, bottom-up parsing membangun struktur sintaks dari bagian paling kecil menuju struktur paling umum. Pendekatan ini banyak digunakan pada parser berbasis shift-reduce, seperti LR parser, karena parser melakukan proses pengurangan token atau kelompok simbol menjadi aturan produksi tertentu sampai seluruh input dapat direduksi menjadi simbol awal grammar.

2.3. Parser Tree

Parse tree adalah representasi berbentuk pohon yang menunjukkan bagaimana suatu input diturunkan berdasarkan aturan grammar. Dalam parse tree, akar pohon merepresentasikan simbol awal grammar, node internal merepresentasikan simbol non-terminal, sedangkan daun pohon merepresentasikan simbol terminal atau token yang berasal dari input.

OpenDSA menjelaskan bahwa grammar digunakan untuk menyatakan sintaks bahasa pemrograman, dan parse tree memperlihatkan proses derivasi dari simbol awal menuju token-token yang membentuk program. Parse tree berbeda dari Abstract Syntax Tree (AST) karena parse tree masih mempertahankan detail grammar secara lebih lengkap, termasuk node non-terminal dan token terminal yang muncul dalam proses parsing.

Parse tree dapat dibangun melalui dua arah, sesuai dengan pendekatan parsing yang digunakan. Pada top-down parsing, parse tree dibentuk dari akar menuju daun. Proses dimulai dari simbol awal grammar, kemudian parser memilih aturan produksi yang sesuai

untuk menurunkan simbol tersebut menjadi simbol-simbol yang lebih kecil hingga akhirnya mencapai token terminal.

Sebaliknya, pada bottom-up parsing, parse tree secara konseptual dibangun dari daun menuju akar. Parser memulai dari token-token terminal pada input, lalu mengelompokkan token tersebut menjadi non-terminal yang lebih besar berdasarkan aturan produksi grammar. Proses ini berlanjut hingga seluruh token berhasil direduksi menjadi simbol awal grammar. Walaupun arah pembangunannya berbeda, hasil akhirnya tetap merepresentasikan hubungan hirarkis antara simbol grammar dan token input.

2.4. Recursive Descent

Recursive descent adalah salah satu teknik parsing top-down. Pada pendekatan ini, struktur grammar diterjemahkan langsung menjadi struktur program. Setiap simbol non-terminal dalam grammar biasanya direpresentasikan oleh satu fungsi atau prosedur parser. Fungsi tersebut bertanggung jawab untuk mengenali pola token yang sesuai dengan aturan produksi milik non-terminal tersebut. Apabila pada sisi kanan aturan produksi terdapat simbol non-terminal lain, fungsi parser akan memanggil fungsi lain yang bersesuaian. Apabila terdapat simbol terminal, fungsi parser akan mencocokkan terminal tersebut dengan token input yang sedang dibaca.

Dalam recursive descent parsing, parser membaca input secara berurutan dari kiri ke kanan. Untuk menentukan aturan produksi mana yang harus digunakan, parser dapat menggunakan token yang sedang dibaca atau beberapa token berikutnya sebagai lookahead. Lookahead membantu parser memilih cabang grammar yang tepat ketika terdapat lebih dari satu kemungkinan produksi. Apabila token yang ditemukan sesuai dengan aturan grammar, parser akan mengonsumsi token tersebut dan melanjutkan ke token berikutnya. Sebaliknya, apabila token tidak sesuai dengan terminal atau pola produksi yang diharapkan, parser dapat melaporkan syntax error. Oleh karena itu, recursive descent tidak hanya digunakan untuk membangun struktur sintaks, tetapi juga untuk mendeteksi kesalahan sintaks pada program sumber.

BAB III

PERANCANGAN DAN IMPLEMENTASI

Implementasi parser dibangun di atas keluaran lexer yang telah diimplementasikan pada Milestone 1. Lexer tetap bertugas mengubah source code Arion menjadi deretan token, sedangkan parser pada Milestone 2 bertugas menerima deretan token tersebut, memeriksa kesesuaiannya terhadap grammar bahasa Arion, dan membangun representasi struktur program dalam bentuk parse tree.

Sebelum diproses oleh parser, token komentar diabaikan karena tidak berpengaruh terhadap struktur sintaks program, sedangkan token tidak dikenal diperlakukan sebagai lexical error. Parse tree direpresentasikan menggunakan struktur `ParseNode`, yang menyimpan label node, token terminal apabila ada, serta daftar child node. Proses parsing dijalankan oleh kelas `Parser` dengan pendekatan recursive descent, yaitu setiap aturan grammar utama diimplementasikan sebagai fungsi parser yang saling memanggil sesuai struktur produksi grammar.

3.1. Gambaran Umum Program

Secara garis besar, gambaran umum program kami sebagai berikut:

1. **Lexical Analysis:** source code Arion di-scan oleh lexer yang telah dibuat pada Milestone 1 untuk menghasilkan deretan token lengkap dengan informasi jenis token, nilai lexeme, serta posisi baris dan kolom.
2. **Token Filtering:** sebelum diberikan kepada parser, token komentar (COMMENT) diabaikan karena tidak mempengaruhi struktur sintaks program. Apabila lexer menghasilkan token tidak dikenal (UNKNOWN), proses dihentikan dan program menampilkan lexical error.
3. **Syntax Analysis:** deretan token yang sudah difilter diberikan kepada objek `Parser`. `Parser` kemudian memeriksa kesesuaian urutan token terhadap grammar bahasa Arion menggunakan pendekatan recursive descent.
4. **Representasi Parse Tree:** setiap simpul dalam parse tree direpresentasikan menggunakan struktur `ParseNode`. Setiap node menyimpan label, token terminal apabila node tersebut merepresentasikan token, serta daftar child node.

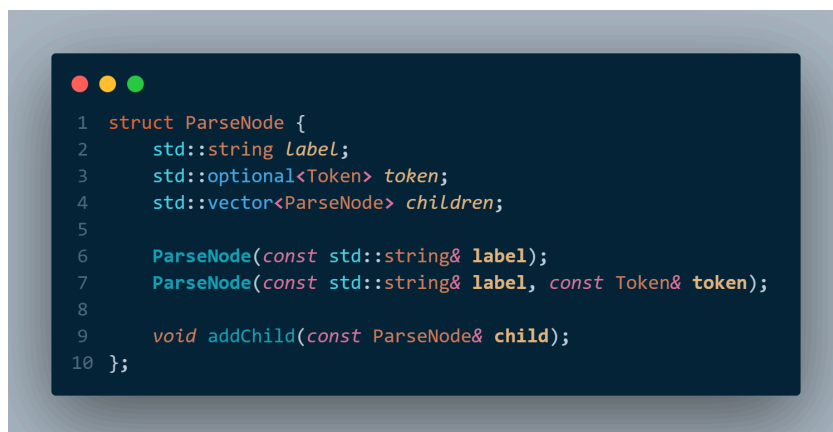
Dengan struktur ini, susunan program dapat ditelusuri dan ditampilkan dalam bentuk tree yang mudah dibaca.

5. Error Handling: jika ditemukan kesalahan sintaks, parser akan melempar `ParseError` dengan pesan yang informatif, mencakup posisi baris dan kolom, token yang ditemukan, serta token atau struktur yang diharapkan.

Dengan arsitektur ini, parser dapat memeriksa apakah susunan token valid menurut grammar Arion dan menghasilkan parse tree yang merepresentasikan struktur sintaks program.

3.2. Representasi Parse Tree

Untuk merepresentasikan parse tree yang dihasilkan oleh parser, kami menggunakan struktur `ParseNode` yang didefinisikan pada file `include/parser.hpp`. Struktur ini menjadi representasi utama untuk seluruh simpul pada parse tree, baik simpul non-terminal maupun terminal. Setiap `ParseNode` menyimpan tiga komponen utama, yaitu label, token, dan children. Atribut label digunakan untuk menyimpan nama node yang akan ditampilkan pada parse tree, misalnya `<program>`, `<declaration-part>`, atau `ident(x)`. Atribut token bersifat opsional dan digunakan apabila node tersebut merepresentasikan token terminal dari lexer. Sementara itu, atribut children menyimpan daftar anak dari node tersebut, sehingga hubungan hirarkis antar bagian grammar dapat dibentuk.

A screenshot of a code editor showing the definition of the `ParseNode` struct in C++. The code is as follows:

```
1 struct ParseNode {
2     std::string label;
3     std::optional<Token> token;
4     std::vector<ParseNode> children;
5
6     ParseNode(const std::string& label);
7     ParseNode(const std::string& label, const Token& token);
8
9     void addChild(const ParseNode& child);
10 };
```

Gambar Struktur `ParseNode`

Pada implementasi, program kami tidak membedakan parse tree ke dalam kelas turunan seperti `TerminalNode` dan `NonTerminalNode`. Perbedaan antara terminal dan

non-terminal direpresentasikan melalui isi `ParseNode`. Node non-terminal biasanya hanya memiliki label dan daftar children, sedangkan node terminal dibuat dari token lexer menggunakan fungsi `makeTerminalNode()`. Fungsi tersebut mengubah token menjadi label yang mudah dibaca melalui `formatToken()`, lalu membungkusnya dalam `ParseNode`.

Proses pembentukan parse tree dilakukan secara bertahap oleh fungsi-fungsi parser. Setiap fungsi parsing membuat sebuah `ParseNode` sesuai non-terminal yang sedang diproses, kemudian menambahkan child node menggunakan method `addChild()`. Sebagai contoh, `parseProgram()` membuat node `<program>`, lalu menambahkan hasil dari `parseProgramHeader()`, `parseDeclarationPart()`, `parseCompoundStatement()`, dan token period sebagai anak-anaknya. Dengan pola tersebut, struktur parse tree yang dihasilkan mengikuti aturan produksi grammar secara langsung.

A screenshot of a code editor showing two C++ functions for parsing. The first function, `parseProgram()`, creates a `ParseNode` for `<program>` and adds four children: `parseProgramHeader()`, `parseDeclarationPart()`, `parseCompoundStatement()`, and a terminal node for the period token. The second function, `parseProgramHeader()`, creates a `ParseNode` for `<program-header>` and adds three children: terminal nodes for `PROGRAMSY`, `IDENT`, and `SEMICOLON`.

```
1 ParseNode Parser::parseProgram() {
2     ParseNode node("<program>");
3
4     node.addChild(parseProgramHeader());
5     node.addChild(parseDeclarationPart());
6     node.addChild(parseCompoundStatement());
7     node.addChild(makeTerminalNode(consume(TokenType::PERIOD, "period")));
8
9     return node;
10 }
11
12 ParseNode Parser::parseProgramHeader() {
13     ParseNode node("<program-header>");
14
15     node.addChild(makeTerminalNode(consume(TokenType::PROGRAMSY, "programsy")));
16     node.addChild(makeTerminalNode(consume(TokenType::IDENT, "ident")));
17     node.addChild(makeTerminalNode(consume(TokenType::SEMICOLON, "semicolon")));
18
19     return node;
20 }
```

Gambar Cuplikan Kode Pembentukan Node Parse Tree

Untuk menampilkan parse tree, program menggunakan fungsi `formatParseTree()` pada file `src/formatter.cpp`. Fungsi ini menelusuri struktur `ParseNode` secara rekursif dan mengubahnya menjadi representasi tree berbasis teks yang dapat ditampilkan di terminal maupun disimpan ke file output.

3.3. Struktur Direktori Program

Struktur directory program kami untuk milestone 2 adalah seperti berikut

SMG-Tubes-IF2224-2026/

```
|─ Makefile           # Skrip untuk kompilasi
|─ README.md         # Dokumentasi umum proyek
|─ bin/              # Executable File
|─ build/            # Folder file hasil kompilasi
|─ doc/              # Dokumentasi
|─ include/          # Folder header file
|   |─ cli.hpp        # Deklarasi antarmuka
command-line
|   |─ driver.hpp     # Deklarasi pengatur alur program
|   |─ filehandler.hpp # Deklarasi utilitas pembacaan
file
|   |─ formatter.hpp  # Deklarasi formatter output
|   |─ lexer.hpp      # Deklarasi lexer
|   |─ parser.hpp     # Deklarasi parser
|─ src/              # Folder Source Code
|   |─ main.cpp       # Program utama
|   |─ cli.cpp        # Implementasi cli
|   |─ driver.cpp     # Pengatur eksekusi program
|   |─ filehandler.cpp # Pembacaan & pengelolaan file
|   |─ formatter.cpp  # Format keluaran program
|   |─ lexer.cpp      # Implementasi lexical analyzer
```



```

|   └─ parser.cpp           # Implementasi utama syntax
analyzer

|   └─ parser_declaration.cpp # Parsing bagian
deklarasi

|   └─ parser_expression.cpp  # Parsing ekspresi

|   └─ parser_statement.cpp   # Parsing statement

└─ test/                     # Folder pengujian program
    └─ milestone-1/          # Test case untuk milestone 1
        └─ input/            # File input pengujian lexer
            └─ output/        # File output hasil lexer
        └─ milestone-2/      # Test case untuk milestone 2
            └─ input/         # File input pengujian parser
                └─ output/    # File output hasil parser

```

Struktur direktori program disusun secara modular agar proses pengembangan, kompilasi, pengujian, dan dokumentasi dapat dilakukan dengan lebih rapi. Pada root project terdapat Makefile yang digunakan untuk mengkompilasi program, README.md sebagai dokumentasi umum proyek, serta beberapa folder utama seperti src, include, bin, build, doc, dan test.

Folder src berisi seluruh implementasi source code utama program. File main.cpp berperan sebagai titik awal eksekusi, sedangkan driver.cpp mengatur alur utama program mulai dari pembacaan input hingga pemrosesan lexer dan parser. File lexer.cpp digunakan untuk melakukan lexical analysis, sementara proses syntax analysis pada milestone 2 diimplementasikan melalui beberapa file parser, yaitu parser.cpp, parser_declaration.cpp, parser_expression.cpp, dan parser_statement.cpp. Pemisahan parser ke beberapa file dilakukan agar bagian deklarasi, ekspresi, dan statement dapat dikelola secara lebih terstruktur.

Folder include berisi header file yang mendeklarasikan fungsi, kelas, dan struktur data yang digunakan oleh file implementasi pada folder src. Dengan adanya pemisahan antara file .hpp dan .cpp, antarmuka program menjadi lebih jelas dan source code lebih mudah dikembangkan. Folder bin digunakan untuk menyimpan executable hasil kompilasi,

sedangkan folder build menyimpan file objek dan dependency sementara yang dihasilkan selama proses kompilasi. Folder doc digunakan untuk menyimpan dokumen laporan proyek.

Folder test digunakan untuk menyimpan kebutuhan pengujian program. Pengujian dipisahkan berdasarkan milestone, yaitu milestone-1 untuk pengujian lexer dan milestone-2 untuk pengujian parser. Pada masing-masing milestone terdapat folder input untuk menyimpan file masukan pengujian dan folder output untuk menyimpan hasil keluaran program setelah dijalankan. Struktur ini memudahkan perbandingan hasil antar milestone serta membantu proses validasi program secara bertahap.

3.4. Kelas Parser dan Mekanisme Recursive Descent

Kelas Parser menyimpan daftar token dalam atribut tokens dan posisi token yang sedang dibaca dalam atribut pos. Untuk menavigasi token, parser menyediakan beberapa fungsi bantu. Fungsi current() digunakan untuk mengambil token saat ini, sedangkan peek() digunakan untuk melihat token berikutnya tanpa mengonsumsi token tersebut. Fungsi check() dan checkNext() digunakan untuk memeriksa tipe token saat ini atau token berikutnya, sedangkan advance() digunakan untuk mengonsumsi token dan memindahkan posisi pembacaan ke token selanjutnya. Selain itu, terdapat fungsi match() yang mengonsumsi token hanya jika tipe token sesuai, serta consume() yang mewajibkan token tertentu muncul pada posisi saat ini. Apabila token yang ditemukan tidak sesuai dengan yang diharapkan, consume() akan melempar ParseError berisi informasi baris, kolom, token yang ditemukan, dan token yang diharapkan.

```

1  class Parser {
2  public:
3      Parser(const std::vector<Token>& tokens);
4
5      ParseNode parse();
6
7  private:
8      std::vector<Token> tokens;
9      size_t pos;
10
11     const Token& current() const;
12     const Token& peek(size_t offset = 1) const;
13
14     bool isAtEnd() const;
15     bool check(TokenType type) const;
16     bool checkNext(TokenType type) const;
17     bool match(TokenType type);
18
19     Token advance();
20     Token consume(TokenType type, const std::string& expected);
21
22     ParseError error(const std::string& message) const;

```

Gambar Kelas Parser dan Handle Error

Proses parsing dimulai melalui fungsi `parse()`. Fungsi ini memanggil `parseProgram()` sebagai start symbol dari grammar Arion. Setelah `parseProgram()` selesai, parser juga memastikan bahwa token berikutnya adalah `END_OF_FILE`, sehingga tidak ada token tambahan yang tersisa setelah struktur program selesai diproses. Fungsi `parseProgram()` kemudian membentuk node `<program>` dan menambahkan hasil parsing dari `parseProgramHeader()`, `parseDeclarationPart()`, `parseCompoundStatement()`, serta token period sebagai anak-anaknya. Dengan pola ini, struktur fungsi parser mengikuti struktur grammar secara langsung.

Untuk bagian deklarasi, parser memisahkan implementasi ke dalam file `parser_declaration.cpp`. Bagian ini menangani aturan grammar seperti `<const-declaration>`, `<type-declaration>`, `<var-declaration>`, `<array-type>`, `<range>`, `<enumerated>`, `<record-type>`, serta deklarasi prosedur dan fungsi. Fungsi `parseDeclarationPart()` mengatur urutan deklarasi agar sesuai dengan grammar, yaitu deklarasi `const`, kemudian `type`, kemudian `var`, lalu subprogram berupa procedure atau function. Jika ditemukan deklarasi yang muncul di luar urutan tersebut, parser akan

menghasilkan error. Pada bagian tipe data, parser juga menggunakan lookahead untuk membedakan apakah sebuah tipe berupa identifier biasa atau range seperti 1..10, karena lexer merepresentasikan range sebagai dua token period berurutan.



```
1 ParseNode Parser::parseConstDeclaration() {
2     ParseNode node("<const-declaration>");
3
4     node.addChild(makeTerminalNode(consume(TokenType::CONSTSY, "constsy")));
5
6     if (!check(TokenType::IDENT)) {
7         throw error("expected at least one constant declaration");
8     }
9
10    do {
11        node.addChild(makeTerminalNode(consume(TokenType::IDENT, "ident")));
12        node.addChild(makeTerminalNode(consume(TokenType::EQL, "eq1")));
13        node.addChild(parseConstant());
14        node.addChild(makeTerminalNode(consume(TokenType::SEMICOLON, "semicolon")));
15    } while (check(TokenType::IDENT));
16
17    return node;
18 }
```

Gambar Cuplikan ParserDeclaration.cpp

Bagian statement diimplementasikan pada file parser_statement.cpp. Fungsi parseStatement() bertugas memilih aturan statement yang sesuai berdasarkan token awal yang sedang dibaca. Statement yang didukung meliputi assignment, procedure/function call, if-statement, case-statement, while-statement, repeat-statement, dan for-statement. Salah satu bagian penting pada statement adalah pembedaan token ident, karena token ini dapat menjadi awal assignment maupun procedure/function call. Untuk itu digunakan fungsi isAssignmentStatementStart() yang melakukan lookahead sampai melewati kemungkinan component variable seperti indeks array dan field record. Jika setelah identifier dan komponen variabel ditemukan token becomes, maka statement diproses sebagai assignment. Jika tidak, identifier pada level statement diproses sebagai procedure/function call.

```

1  ParseNode Parser::parseCompoundStatement() {
2      ParseNode node("<compound-statement>");
3
4      node.addChild(makeTerminalNode(consume(TokenType::BEGINSY, "beginsy")));
5      node.addChild(parseStatementList());
6      node.addChild(makeTerminalNode(consume(TokenType::ENDSY, "endsy")));
7
8      return node;
9  }
10
11 ParseNode Parser::parseStatementList() {
12     ParseNode node("<statement-list>");
13
14     node.addChild(parseStatement());
15
16     while (match(TokenType::SEMICOLON)) {
17         node.addChild(makeTerminalNode(tokens[pos - 1]));
18
19         if (check(TokenType::ENDSY) ||
20             check(TokenType::UNTILSY) ||
21             check(TokenType::END_OF_FILE)) {
22             break;
23         }
24
25         node.addChild(parseStatement());
26     }
27
28     return node;
29 }

```

Gambar Cuplikan ParserStatement.cpp

Bagian ekspresi diimplementasikan pada file parser_expression.cpp. Parser membagi ekspresi menjadi beberapa level, yaitu <expression>, <simple-expression>, <term>, dan <factor>. Pembagian ini digunakan untuk merepresentasikan prioritas operator secara langsung di dalam grammar. Operator relasional seperti eql, neq, gtr, geq, lss, dan leq diproses pada level <expression>. Operator penjumlahan, pengurangan, dan orsy diproses pada level <simple-expression>. Sementara itu, operator perkalian, pembagian, div, mod, dan andsy diproses pada level <term>. Dengan struktur bertingkat ini, operator dengan prioritas lebih tinggi akan dikelompokkan lebih dalam pada parse tree, sehingga ekspresi dapat diparse sesuai aturan precedence.



```

1  ParseNode Parser::parseExpression() {
2      ParseNode node("<expression>");
3
4      node.addChild(parseSimpleExpression());
5
6      if (isRelationalOperator(current().type)) {
7          node.addChild(parseRelationalOperator());
8          node.addChild(parseSimpleExpression());
9      }
10
11     return node;
12 }
13
14 ParseNode Parser::parseSimpleExpression() {
15     ParseNode node("<simple-expression>");
16
17     if (check(TokenType::PLUS) || check(TokenType::MINUS)) {
18         node.addChild(makeTerminalNode(advance()));
19     }
20
21     node.addChild(parseTerm());
22
23     while (isAdditiveOperator(current().type)) {
24         node.addChild(parseAdditiveOperator());
25         node.addChild(parseTerm());
26     }
27
28     return node;
29 }

```

Gambar Cuplikan ParserExpression.cpp

Setiap fungsi parsing menghasilkan sebuah ParseNode. Node tersebut diberi label sesuai non-terminal yang sedang diproses, misalnya <program>, <assignment-statement>, atau <expression>. Apabila parser membaca token terminal, token tersebut dibungkus menjadi node terminal menggunakan fungsi makeTerminalNode(). Node-node yang dihasilkan kemudian disusun menggunakan method addChild(), sehingga terbentuk parse tree yang merepresentasikan struktur sintaks program Arion. Jika seluruh token berhasil diproses tanpa error, parse tree dikembalikan sebagai hasil syntax analyzer. Jika terjadi kesalahan sintaks, parser menghentikan proses dan mengembalikan pesan error yang informatif melalui mekanisme ParseError.

3.5. Perancangan Grammar Arion dan Pemetaan ke Fungsi Parser

Struktur program utama Arion dipetakan melalui fungsi parseProgram(). Fungsi ini merepresentasikan aturan bahwa sebuah program terdiri dari program header, declaration part, compound statement, dan diakhiri dengan period. Bagian program header diproses

oleh `parseProgramHeader()`, sedangkan bagian deklarasi diproses oleh `parseDeclarationPart()`. Dengan demikian, parsing selalu dimulai dari simbol awal `<program>` dan secara bertahap turun ke bagian-bagian penyusunnya.

Pada bagian deklarasi, parser menyediakan fungsi khusus untuk setiap jenis deklarasi. Deklarasi konstanta diproses oleh `parseConstDeclaration()`, deklarasi tipe oleh `parseTypeDeclaration()`, dan deklarasi variabel oleh `parseVarDeclaration()`. Selain itu, subprogram berupa `procedure` dan `function` masing-masing diproses oleh `parseProcedureDeclaration()` dan `parseFunctionDeclaration()`, yang dibungkus oleh `parseSubprogramDeclaration()`. Fungsi `parseDeclarationPart()` memastikan bahwa urutan deklarasi mengikuti spesifikasi `grammar`, yaitu deklarasi `const`, kemudian `type`, kemudian `var`, lalu deklarasi `procedure` atau `function`. Jika ditemukan deklarasi yang tidak sesuai urutan tersebut, parser akan menghasilkan `error`.

Kami juga memetakan Arion ke beberapa fungsi parser. Fungsi `parseType()` menjadi titik masuk untuk mengenali berbagai bentuk tipe, seperti `identifier` tipe bawaan atau tipe buatan pengguna, `array`, `range`, `enumerated`, dan `record`. Tipe `array` diproses oleh `parseArrayType()`, `range` oleh `parseRange()`, `enumerated` oleh `parseEnumerated()`, dan `record` oleh `parseRecordType()`. Untuk `record`, `field-field` di dalamnya diproses melalui `parseFieldList()` dan `parseFieldPart()`. Pada bagian ini parser menggunakan `lookahead` untuk membedakan `identifier` biasa dengan `range`, karena `range` direpresentasikan oleh dua token `period` berurutan, misalnya `1..10`.

Bagian `statement` dipetakan melalui fungsi `parseCompoundStatement()`, `parseStatementList()`, dan `parseStatement()`. `Compound statement` mengenali blok yang diawali token `beginsy` dan diakhiri token `endsy`. Di dalam blok tersebut, `parseStatementList()` memproses daftar `statement` yang dipisahkan oleh `semicolon`. Fungsi `parseStatement()` kemudian memilih jenis `statement` berdasarkan token yang sedang dibaca. `Statement` yang didukung meliputi `assignment statement`, `if statement`, `case statement`, `while statement`, `repeat statement`, `for statement`, serta `procedure/function call`.

Untuk `statement` yang diawali `ident`, parser perlu melakukan pembedaan karena `identifier` dapat menjadi awal `assignment` maupun `procedure/function call`. Fungsi `isAssignmentStatementStart()` digunakan untuk melakukan `lookahead` terhadap token-token setelah `identifier`. Jika setelah `identifier` dan kemungkinan `component`

variable seperti array index atau field record ditemukan token becomes, maka statement diproses sebagai assignment melalui `parseAssignmentStatement()`. Jika tidak, identifier pada posisi statement diproses sebagai procedure/function call melalui `parseProcedureOrFunctionCall()`.

Bagian variable juga memiliki pemetaan khusus. Fungsi `parseVariable()` mengenali identifier yang dapat diikuti oleh nol atau lebih component variable. Component variable diproses oleh `parseComponentVariable()`, yang dapat berupa akses array menggunakan pasangan lbrack dan rbrack, atau akses field record menggunakan token period diikuti identifier. Dengan desain ini, parser dapat mengenali bentuk variable seperti identifier biasa, elemen array, field record, maupun kombinasi keduanya.

Bagian expression dipetakan ke fungsi `parseExpression()`, `parseSimpleExpression()`, `parseTerm()`, dan `parseFactor()`. Pembagian ini mengikuti tingkat prioritas operator. Operator relasional diproses pada level `parseExpression()`, operator penjumlahan, pengurangan, dan orsy diproses pada level `parseSimpleExpression()`, sedangkan operator perkalian, pembagian, idiv, imod, dan andsy diproses pada level `parseTerm()`. Unit terkecil expression diproses oleh `parseFactor()`, yang dapat berupa identifier, literal, ekspresi dalam tanda kurung, operator notsy, procedure/function call, atau variable.

Secara ringkas, pemetaan grammar ke fungsi parser dapat dilihat sebagai berikut:

<code><program></code>	<code>-> parseProgram()</code>
<code><program-header></code>	<code>-> parseProgramHeader()</code>
<code><declaration-part></code>	<code>-> parseDeclarationPart()</code>
<code><const-declaration></code>	<code>-> parseConstDeclaration()</code>
<code><type-declaration></code>	<code>-> parseTypeDeclaration()</code>
<code><var-declaration></code>	<code>-> parseVarDeclaration()</code>
<code><type></code>	<code>-> parseType()</code>
<code><array-type></code>	<code>-> parseArrayType()</code>
<code><range></code>	<code>-> parseRange()</code>
<code><enumerated></code>	<code>-> parseEnumerated()</code>
<code><record-type></code>	<code>-> parseRecordType()</code>

<subprogram-declaration>	->
parseSubprogramDeclaration()	
<procedure-declaration>	->
parseProcedureDeclaration()	
<function-declaration>	->
parseFunctionDeclaration()	
<compound-statement>	-> parseCompoundStatement()
<statement-list>	-> parseStatementList()
<statement>	-> parseStatement()
<variable>	-> parseVariable()
<assignment-statement>	->
parseAssignmentStatement()	
<if-statement>	-> parseIfStatement()
<case-statement>	-> parseCaseStatement()
<while-statement>	-> parseWhileStatement()
<repeat-statement>	-> parseRepeatStatement()
<for-statement>	-> parseForStatement()
<procedure/function-call>	->
parseProcedureOrFunctionCall()	
<expression>	-> parseExpression()
<simple-expression>	-> parseSimpleExpression()
<term>	-> parseTerm()
<factor>	-> parseFactor()

3.6. Alur Eksekusi Program

Secara garis besar, alur eksekusi syntax analyzer pada program adalah sebagai berikut:

1. Program membaca source code Arion dari file input yang dipilih pengguna melalui CLI.

2. Jika pengguna memilih folder milestone-1, program hanya menjalankan lexical analyzer melalui `runLexer()`. Jika pengguna memilih folder milestone-2, program menjalankan syntax analyzer melalui `runSyntaxAnalyzer()`.
3. Pada `runSyntaxAnalyzer()`, source code diberikan kepada objek `Lexer` untuk diubah menjadi deretan token menggunakan fungsi `getNextToken()`.
4. Setiap token hasil lexer difilter sebelum masuk ke parser. Token `COMMENT` diabaikan, sedangkan token `UNKNOWN` langsung menghasilkan lexical error berisi informasi baris, kolom, dan nilai token bermasalah.
5. Token yang valid dikumpulkan ke dalam vector `tokens` hingga lexer menghasilkan token `END_OF_FILE`.
6. Setelah token terkumpul, program membuat objek `Parser` dan memanggil fungsi `parse()` sebagai entry point syntax analysis.
7. Parser memulai proses dari simbol awal `<program>` melalui `parseProgram()`, lalu memanggil fungsi-fungsi parser lain sesuai grammar Arion untuk membangun parse tree.
8. Setiap non-terminal direpresentasikan sebagai `ParseNode`, sedangkan token terminal dibungkus menggunakan `makeTerminalNode()`. Node-node tersebut disusun dengan `addChild()`.
9. Jika ditemukan token yang tidak sesuai grammar, parser melempar `ParseError` dan pesan error tersebut dikembalikan sebagai output.
10. Jika parsing berhasil, parse tree diformat oleh `formatParseTree()`, ditampilkan ke terminal, dan disimpan ke file output.

BAB IV

PENGUJIAN

4.1. Pengujian 1

Input
<pre> program Hello; var a, b: integer; begin a := 5; b := a + 10; writeln('Result = ', b); end.</pre>
Output
<pre> <program> ├─ <program-header> ├─ programsy ├─ ident(Hello) └─ semicolon ├─ <declaration-part> └─ <var-declaration> ├─ varsy ├─ <identifier-list> ├─ ident(a) ├─ comma └─ ident(b) ├─ colon ├─ <type> └─ ident(integer) └─ semicolon ├─ <compound-statement> ├─ beginsy └─ <statement-list> └─ <assignment-statement></pre>

<pre> └─ semicolon └─ endsy └─ period </pre>
Penjelasan
Testcase ini menguji bentuk dasar program Arion yang valid, dimulai dari <code>program-header</code>, bagian deklarasi variabel, lalu <code>compound-statement</code> yang diakhiri dengan tanda titik. Pada bagian deklarasi, parser harus mengenali <code>var-declaration</code> dengan <code>identifier-list</code> berisi dua identifier, yaitu <code>a</code> dan <code>b</code>, yang memiliki tipe <code>integer</code>. Pada bagian <code>statement</code>, testcase ini menguji assignment sederhana <code>a := 5</code>, assignment dengan ekspresi aritmetika <code>b := a + 10</code>, serta pemanggilan prosedur <code>writeln</code>. Output parse tree menunjukkan bahwa parser berhasil membentuk struktur <code>assignment-statement</code>, <code>expression</code>, <code>simple-expression</code>, <code>term</code>, dan <code>factor</code> sesuai grammar, termasuk mengenali parameter <code>writeln</code> berupa string dan variabel.

4.2. Pengujian 2

Input
<pre> program Declarations; const Limit == 10; Letter == 'z'; Title == 'ok'; type Index == 1..10; Day == (Mon, Tue, Wed); Scores == array[Index] of integer; Person == record age: integer; grade: char; end; var i: Index; p: Person; </pre>

```
begin
```

```
    i := Limit;
```

```
    p.age := i;
```

```
end.
```

Output

```
<program>
```

```
├─ <program-header>
```

```
|   ├─ programsy
```

```
|   ├─ ident(Declarations)
```

```
|   └─ semicolon
```

```
├─ <declaration-part>
```

```
|   ├─ <const-declaration>
```

```
|       ├─ constsy
```

```
|       ├─ ident(Limit)
```

```
|       ├─ eql
```

```
|       ├─ <constant>
```

```
|           └─ intcon(10)
```

```
|       ├─ semicolon
```

```
|       ├─ ident(Letter)
```

```
|       ├─ eql
```

```
|       ├─ <constant>
```

```
|           └─ charcon('z')
```

```
|       ├─ semicolon
```

```
|       ├─ ident(Title)
```

```
|       ├─ eql
```

```
|       ├─ <constant>
```

```
|           └─ string('ok')
```

```
|       └─ semicolon
```

```
|   └─ <type-declaration>
```

```
|       ├─ typesy
```

```
|       ├─ ident(Index)
```

```
|       ├─ eql
```

```
|       ├─ <type>
```

```
|           └─ <range>
```

```
|               └─ <constant>
```

```
|                   └─ intcon(1)
```

```
|               └─ period
```

```
|               └─ period
```

```
|               └─ <constant>
```

			└─ intcon(10)
			└─ semicolon
			└─ ident(Day)
			└─ eql
			└─ <type>
			└─ ┌─ <enumerated>
			└─ └─ lparent
			└─ └─ ident(Mon)
			└─ └─ comma
			└─ └─ ident(Tue)
			└─ └─ comma
			└─ └─ ident(Wed)
			└─ └─ rparent
			└─ semicolon
			└─ ident(Scores)
			└─ eql
			└─ <type>
			└─ ┌─ <array-type>
			└─ └─ arraysy
			└─ └─ lbrack
			└─ └─ ident(Index)
			└─ └─ rbrack
			└─ └─ ofsy
			└─ └─ <type>
			└─ └─ ┌─ ident(integer)
			└─ └─ semicolon
			└─ └─ ident(Person)
			└─ └─ eql
			└─ └─ <type>
			└─ └─ ┌─ <record-type>
			└─ └─ └─ recordsy
			└─ └─ └─ <field-list>
			└─ └─ └─ ┌─ <field-part>
			└─ └─ └─ └─ ┌─ <identifier-list>
			└─ └─ └─ └─ └─ ┌─ ident(age)
			└─ └─ └─ └─ └─ └─ colon
			└─ └─ └─ └─ └─ └─ ┌─ <type>
			└─ └─ └─ └─ └─ └─ └─ ┌─ ident(integer)
			└─ └─ └─ └─ └─ └─ └─ └─ semicolon
			└─ └─ └─ └─ └─ └─ └─ └─ <field-part>
			└─ └─ └─ └─ └─ └─ └─ └─ <identifier-list>

				└ becomes
				└ <expression>
				└ <simple-expression>
				└ <term>
				└ <factor>
				└ <variable>
				└ ident(i)
				└ semicolon
				└ endsy
				└ period
Penjelasan				
Testcase ini menguji bagian deklarasi yang lebih lengkap sesuai urutan pada spesifikasi, yaitu const-declaration, type-declaration, dan var-declaration. Pada deklarasi konstanta, parser diuji untuk mengenali beberapa jenis constant, yaitu integer, character, dan string. Bagian type declaration menguji beberapa bentuk tipe yang ada pada spesifikasi, seperti range pada <code>Index == 1..10</code>, enumerated pada <code>Day</code>, array-type pada <code>Scores</code>, serta record-type pada <code>Person</code>. Selain itu, pada bagian statement, parser juga diuji untuk mengenali assignment ke variabel biasa dan component variable record seperti <code>p.age</code>.				

4.3. Pengujian 3

Input
<pre> program Subprograms; procedure Show(x: integer; y: array[1..3] of integer); var local: integer; begin local := x; writeln(local); end; function Next(n: integer): integer; begin Next := n + 1; </pre>

```

end;

begin
    Show(Next(1), 2);
end.

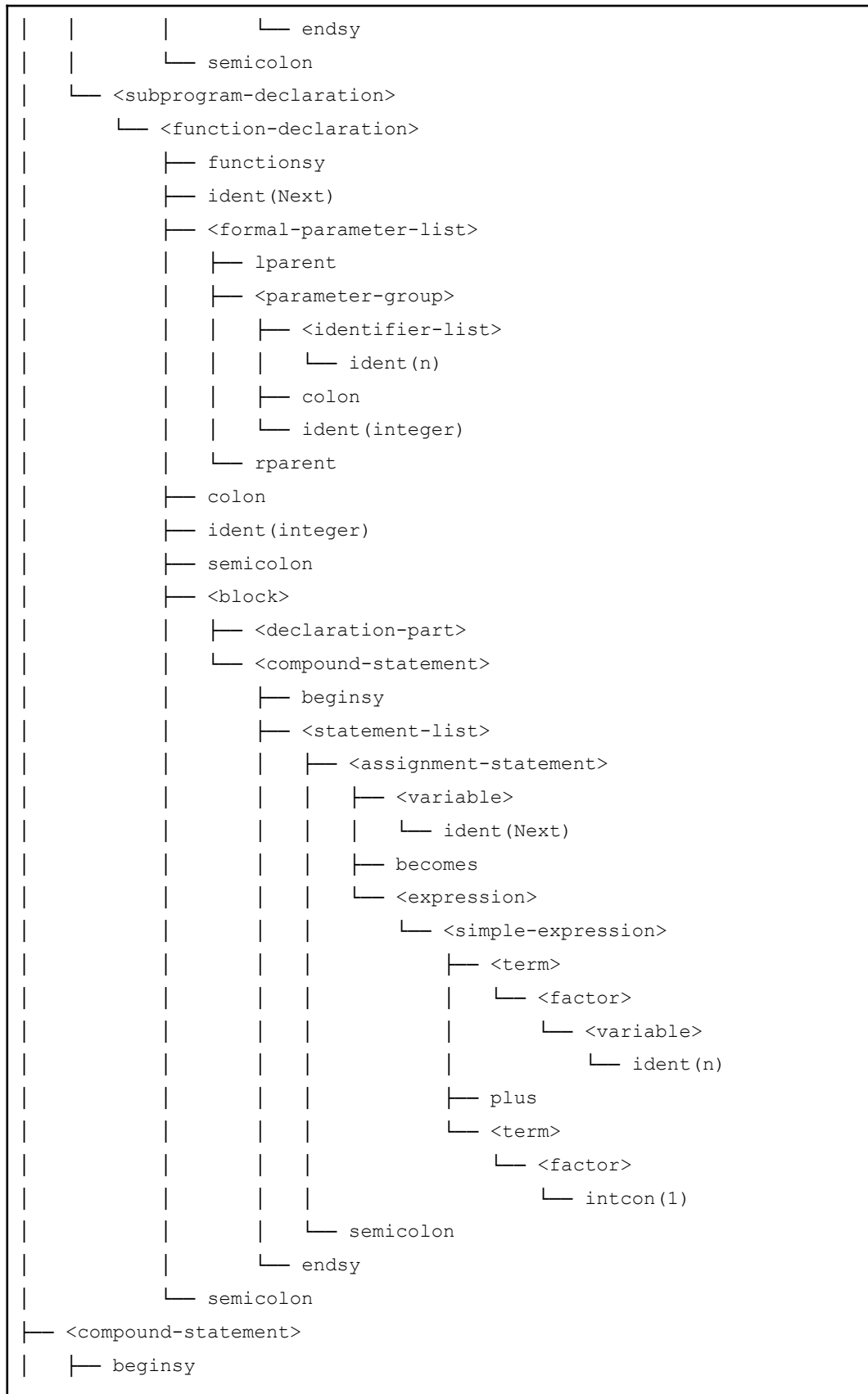
```

Output

```

<program>
├─ <program-header>
|   ├─ programsy
|   └─ ident(Subprograms)
└─ semicolon
├─ <declaration-part>
|   └─ <subprogram-declaration>
|       └─ <procedure-declaration>
|           ├─ proceduresy
|           ├─ ident(Show)
|           └─ <formal-parameter-list>
|               └─ lparent
|                   └─ <parameter-group>
|                       └─ <identifier-list>
|                           └─ ident(x)
|                               └─ colon
|                                   └─ ident(integer)
|                                       └─ semicolon
|                                           └─ <parameter-group>
|                                               └─ <identifier-list>
|                                                   └─ ident(y)
|                                                       └─ colon
|                                                           └─ <array-type>
|                                                               └─ arraysy
|                                                                   └─ lbrack
|                                                                       └─ <range>
|                                                                           └─ <constant>
|                                                                               └─ intcon(1)
|                                                                                   └─ period
|                                                                                       └─ period
|                                                                                           └─ <constant>
|                                                                                               └─ intcon(3)
|                                                                                                   └─ rbrack

```

```

|   └─ <statement-list>
|   └─ <procedure/function-call>
|       └─ ident (Show)
|           └─ lparent
|               └─ <parameter-list>
|                   └─ <expression>
|                       └─ <simple-expression>
|                           └─ <term>
|                               └─ <factor>
|                                   └─ <procedure/function-call>
|                                       └─ ident (Next)
|                                           └─ lparent
|                                               └─ <parameter-list>
|                                                   └─ <expression>
|                                                       └─ <simple-expression>
|                                                           └─ <term>
|                                                               └─ <factor>
|                                                                   └─ intcon (1)
|                                                                       └─ rparent
|                                                                           └─ comma
|                                                                               └─ <expression>
|                                                                                   └─ <simple-expression>
|                                                                                       └─ <term>
|                                                                                           └─ <factor>
|                                                                                               └─ intcon (2)
|                                                                                                   └─ rparent
|                                                                                                       └─ semicolon
|                                                                                                           └─ endsy
|                                                                                                               └─ period

```

Penjelasan

Testcase ini berfokus pada deklarasi subprogram, yaitu procedure dan function. Parser harus mengenali procedure-declaration untuk Show yang memiliki formal parameter list, termasuk parameter biasa bertipe integer dan parameter bertipe array `array[1..3] of integer`.

Selain procedure, testcase ini juga menguji function-declaration bernama Next yang memiliki parameter dan tipe return integer. Di dalam blok function, terdapat assignment ke nama fungsi sebagai bentuk pengembalian nilai. Pada program utama, pemanggilan `Show (Next (1), 2)` juga menguji bahwa parser dapat mengenali function call sebagai factor di dalam parameter procedure call.

4.4. Pengujian 4

Input
<pre> program BranchLoop; var x, y: integer; begin if x > 0 then y := 1 else y := 0; while y < 10 do y := y + 1; repeat y := y - 1 until y == 0; end. </pre>
Output
<pre> <program> ├─ <program-header> ├─ programsy ├─ ident(BranchLoop) └─ semicolon ├─ <declaration-part> └─ <var-declaration> ├─ varsy ├─ <identifier-list> ├─ ident(x) ├─ comma └─ ident(y) ├─ colon ├─ <type> └─ ident(integer) └─ semicolon ├─ <compound-statement> ├─ beginsy ├─ <statement-list> ├─ <if-statement> └─ ifsy </pre>

					└─ <simple-expression>
					└─ <term>
					└─ <factor>
					└─ <variable>
					└─ ident(y)
					└─ <relational-operator>
					└─ eql
					└─ <simple-expression>
					└─ <term>
					└─ <factor>
					└─ intcon(0)
					└─ semicolon
					└─ endsy
					└─ period

Penjelasan

Testcase ini menguji beberapa bentuk statement kontrol alur yang tercantum pada spesifikasi, yaitu if-statement, while-statement, dan repeat-statement. Pada statement if $x > 0$ then ... else ..., parser harus mengenali ekspresi relasional dengan operator $>$ serta dua cabang assignment untuk bagian then dan else.

Statement while $y < 10$ do $y := y + 1$ menguji pengenalan kondisi perulangan dan body berupa assignment dengan ekspresi penjumlahan. Sementara itu, repeat ... until $y == 0$ menguji struktur perulangan yang memiliki statement-list sebelum kondisi akhir, serta ekspresi relasional menggunakan operator equality $==$.

4.5. Pengujian 5

Input
<pre> program CaseFor; var i, result: integer; begin for i := 10 downto 1 do result := result + i; case result of </pre>

```

0: writeln('zero');
1, 2: writeln('small');
end;
end.

```

Output

```

<program>
├─ <program-header>
|   ├─ programsy
|   └─ ident(CaseFor)
└─ semicolon
├─ <declaration-part>
|   └─ <var-declaration>
|       ├─ varsy
|       └─ <identifier-list>
|           ├─ ident(i)
|           └─ comma
|               └─ ident(result)
└─ colon
    ├─ <type>
    │   └─ ident(integer)
    └─ semicolon
├─ <compound-statement>
|   ├─ beginsy
|   └─ <statement-list>
|       ├─ <for-statement>
|       │   ├─ forsy
|       │   └─ ident(i)
|       │       └─ becomes
|       │           └─ <expression>
|       │               └─ <simple-expression>
|       │                   └─ <term>
|       │                       └─ <factor>
|       │                           └─ intcon(10)
|       └─ downtosy
|           └─ <expression>
|               └─ <simple-expression>
|                   └─ <term>
|                       └─ <factor>
|                           └─ intcon(1)

```


					└─ <case-block>
					└─ <constant>
					└─┬─ intcon(1)
					└─ comma
					└─ <constant>
					└─┬─ intcon(2)
					└─ colon
					└─ <procedure/function-call>
					└─┬─ ident(writeln)
					└─┬─ lparent
					└─┬─ <parameter-list>
					└─┬─┬─ <expression>
					└─┬─┬─┬─ <simple-expression>
					└─┬─┬─┬─┬─ <term>
					└─┬─┬─┬─┬─┬─ <factor>
					└─┬─┬─┬─┬─┬─┬─ string('small')
					└─┬─ rparent
					└─ semicolon
					└─ endsy
					└─ semicolon
					└─ endsy
					└─ period

Penjelasan

Testcase ini menguji statement for dan case. Pada `for i := 10 downto 1 do ...`, parser harus mengenali identifier loop, assignment awal, arah iterasi `downto`, ekspresi batas akhir, serta statement yang dijalankan di dalam loop.

Bagian `case result of` menguji struktur case-statement dengan beberapa case-block. Testcase ini juga memastikan parser dapat mengenali label case berupa konstanta tunggal seperti `0` maupun daftar konstanta seperti `1, 2`, kemudian menghubungkannya dengan statement pemanggilan prosedur `writeln`.

4.6. Pengujian 6

Input
program Components;

```

type
  Matrix == array[1..3] of array[1..3] of integer;
  Point == record
    x: integer;
    y: integer;
  end;

var
  m: Matrix;
  p: Point;

begin
  m[1][2] := p.x + p.y;
  p.y := m[2][3];
end.

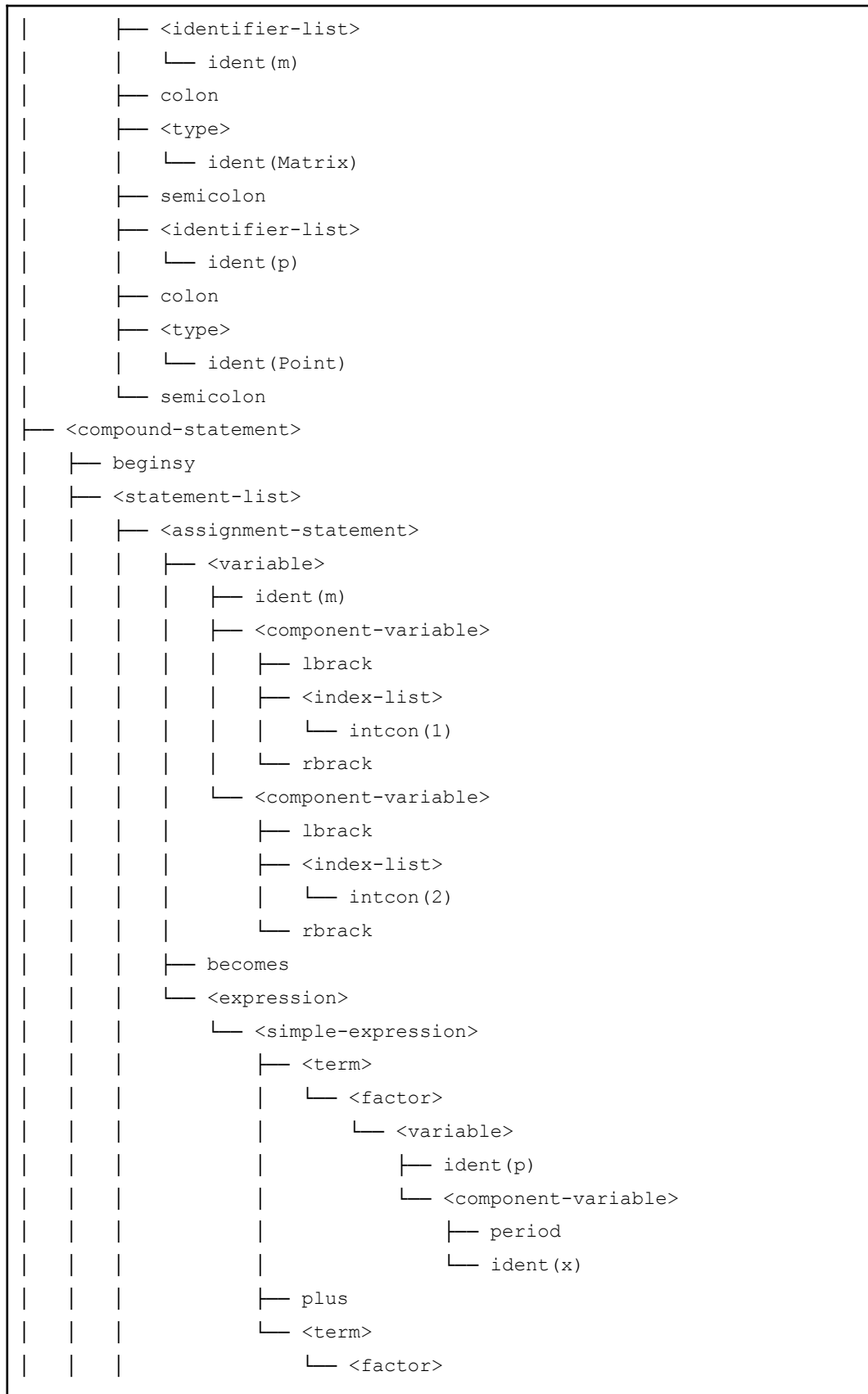
```

Output

```

<program>
├─ <program-header>
|   ├─ programsy
|   └─ ident(Components)
└─ <declaration-part>
    ├─ <type-declaration>
    |   ├─ typesy
    |   └─ ident(Matrix)
    └─ eql
        └─ <type>
            └─ <array-type>
                ├─ arraysy
                └─ lbrack
                    └─ <range>
                        └─ <constant>
                            └─ intcon(1)
                                └─ period
                                    └─ period
                                        └─ <constant>
                                            └─ intcon(3)
                                                └─ rbrack

```

akses field record seperti `p.x` dan `p.y`, baik sebagai tujuan assignment maupun sebagai bagian dari ekspresi.

4.7. Pengujian 7

Input
<pre> program Expressions; var a, b, c: integer; begin a := -1 + 2 * (3 + 4); b := not a or b and c; c := a div b mod c; end.</pre>
Output
<pre> <program> ├─ <program-header> ├─ programsy └─ ident(Expressions) └─ semicolon ├─ <declaration-part> └─ <var-declaration> ├─ varsy └─ <identifier-list> ├─ ident(a) ├─ comma ├─ ident(b) ├─ comma └─ ident(c) └─ colon └─ <type> └─ ident(integer) └─ semicolon └─ <compound-statement></pre>

				└─ <variable>
				└─ ident(a)
				└─ orsy
				└─ <term>
				└─ <factor>
				└─ <variable>
				└─ ident(b)
				└─ <multiplicative-operator>
				└─ andsy
				└─ <factor>
				└─ <variable>
				└─ ident(c)
				└─ semicolon
				└─ <assignment-statement>
				└─ <variable>
				└─ ident(c)
				└─ becomes
				└─ <expression>
				└─ <simple-expression>
				└─ <term>
				└─ <factor>
				└─ <variable>
				└─ ident(a)
				└─ <multiplicative-operator>
				└─ idiv
				└─ <factor>
				└─ <variable>
				└─ ident(b)
				└─ <multiplicative-operator>
				└─ imod
				└─ <factor>
				└─ <variable>
				└─ ident(c)
				└─ semicolon
				└─ endsy
				└─ period
Penjelasan				
Testcase ini berfokus pada parsing ekspresi. Assignment pertama, a := -1 + 2 * (3 + 4), menguji unary minus, operator penjumlahan, operator perkalian, dan penggunaan tanda kurung untuk				

mengelompokkan ekspresi.

Assignment kedua dan ketiga menguji operator logika serta operator aritmetika khusus, yaitu `not`, `or`, `and`, `div`, dan `mod`. Output parse tree memperlihatkan bahwa parser membagi ekspresi ke dalam node `simple-expression`, `term`, dan `factor`, sehingga prioritas operator dapat direpresentasikan sesuai struktur grammar.

4.8. Pengujian 8

Input
<pre>program Calls; begin { comments are ignored } writeln(); writeln('done'); end.</pre>
Output
<pre><program> ├─ <program-header> ├─ programsy └─ ident(Calls) └─ semicolon ├─ <declaration-part> └─ <compound-statement> ├─ beginsy └─ <statement-list> ├─ <procedure/function-call> │ ├─ ident(writeln) │ └─ lpparent │ └─ rparent │ └─ semicolon └─ <procedure/function-call> ├─ ident(writeln) └─ lpparent └─ <parameter-list></pre>

				└ <code><expression></code>
				└ <code><simple-expression></code>
				└ <code><term></code>
				└ <code><factor></code>
				└ <code>string('done')</code>
			└ <code>rparent</code>	
		└ <code>semicolon</code>		
	└ <code>endsy</code>			
└ <code>period</code>				

Penjelasan

Testcase ini menguji program tanpa deklarasi, sehingga declaration-part kosong tetapi tetap valid menurut spesifikasi. Program langsung masuk ke compound-statement yang berisi dua pemanggilan prosedur `writeln`.

Pemanggilan pertama, `writeln()`, menguji procedure call dengan parameter kosong, sedangkan `writeln('done')` menguji procedure call dengan satu parameter berupa string. Testcase ini juga memuat komentar `{ comments are ignored }`, sehingga menunjukkan bahwa komentar telah diabaikan oleh lexer dan tidak muncul sebagai node pada parse tree.

4.9. Pengujian 9

Input
<pre> program BadDeclarationOrder; var x: integer; const y == 1; begin end. </pre>
Output
Syntax error at line 5, column 1: declarations must be ordered as const, type, var, then procedure/function, found constsy (const)

Penjelasan
Testcase ini merupakan kasus tidak valid yang menguji aturan urutan deklarasi pada spesifikasi. Program mendeklarasikan <code>var</code> terlebih dahulu, lalu mencoba mendeklarasikan <code>const</code>, padahal declaration part harus berurutan: <code>const</code>, kemudian <code>type</code>, kemudian <code>var</code>, lalu <code>procedure/function</code>. Karena token <code>const</code> muncul setelah bagian <code>var</code>, parser menolak input dan menghasilkan <code>syntax error</code>. Pesan error menunjukkan lokasi kesalahan pada baris 5 kolom 1, serta menjelaskan bahwa deklarasi harus mengikuti urutan yang telah ditentukan.

4.10. Pengujian 10

Input
<pre>program BadEqual; begin x = 1; end.</pre>
Output
<pre>Lexical error at line 4, column 7: =</pre>
Penjelasan
Testcase ini merupakan kasus error pada tahap lexical analysis. Pada statement <code>x = 1</code>, input menggunakan simbol <code>=</code>, sedangkan berdasarkan spesifikasi token yang valid untuk assignment adalah <code>:=</code>, dan operator equality ditulis sebagai <code>==</code>. Karena karakter <code>=</code> tunggal tidak dikenali sebagai token valid oleh lexer, program tidak sampai ke tahap pembentukan parse tree. Output yang dihasilkan adalah lexical error pada baris 4 kolom 7, yang menunjukkan bahwa kesalahan terjadi sebelum proses syntax analysis dilakukan.

BAB V

KESIMPULAN DAN SARAN

5.1. Kesimpulan

Pada milestone 2 ini, program Arion Compiler telah berhasil dikembangkan dari tahap lexical analysis menuju syntax analysis. Program mampu membaca source code Arion, mengubahnya menjadi deretan token melalui lexer, lalu memeriksa kesesuaian struktur token tersebut terhadap grammar bahasa Arion menggunakan metode Recursive Descent Parser.

Parser yang dibuat telah mampu mengenali berbagai struktur sintaks utama, seperti program header, declaration part, compound statement, deklarasi konstanta, tipe, variabel, procedure, function, statement percabangan, perulangan, assignment, procedure/function call, serta ekspresi. Hasil dari proses parsing direpresentasikan dalam bentuk parse tree, sehingga struktur hierarkis program dapat terlihat dengan jelas.

Berdasarkan hasil pengujian, program dapat menghasilkan parse tree untuk input yang valid dan memberikan pesan kesalahan untuk input yang tidak valid, baik berupa lexical error maupun syntax error. Dengan demikian, program telah memenuhi tujuan milestone 2, yaitu melakukan analisis sintaksis terhadap source code Arion sesuai spesifikasi grammar yang diberikan.

5.2. Saran

Pelaksanaan Milestone 2 Tugas Besar Teori Bahasa Formal dan Automata di Semester II (Genap) Tahun 2025/2026 merupakan pengalaman yang sangat berharga bagi kami. Dari pengalaman ini, kami ingin berbagi beberapa saran kepada pembaca yang mungkin akan menghadapi tugas serupa di masa depan:

Untuk pengembangan selanjutnya, parser dapat dilengkapi dengan mekanisme error recovery agar program tidak langsung berhenti pada kesalahan pertama. Dengan demikian, pengguna dapat memperoleh beberapa pesan kesalahan sekaligus dalam satu kali proses kompilasi.

Selain itu, struktur parse tree yang dihasilkan dapat dikembangkan lebih lanjut agar lebih mudah digunakan pada tahap berikutnya, yaitu semantic analysis. Pengujian juga dapat diperluas dengan lebih banyak variasi kasus, terutama untuk kombinasi grammar yang kompleks, nested statement, dan penggunaan procedure/function yang lebih beragam.

BAB VI

LAMPIRAN

6.1. Lampiran Link Repository Github.

Link : <https://github.com/ucupsss/SMG-Tubes-IF2224-2026>

6.2. Pembagian Tugas

NIM	Nama	Tugas	Persentase Kontribusi (%)
13524014	Yusuf Faishal Listyardi	Implementasi <code>parser.cpp</code> , <code>parser_declaration.c</code> pp dan laporan	25
13524046	Farrell Limjaya	Implementasi <code>parser_statement.cpp</code> , <code>parser_expression.cp</code> p dan laporan	25
13524066	Nathanael Gunawan	Integrasi parser ke cli dan laporan	25
13524070	A. Fawwaz Azam Wicaksono	Revisi lexer, test case, dan laporan	25

REFERENSI

- <https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0264326#pone.0264326.ref001>
- <https://opensa.cs.vt.edu/ODSA/Books/PL/html/Grammars1.html>
- <https://www.cs.odu.edu/~toida/nerzic/390teched/cfl/cfg.html>